UTX. PROYECTO DE APLICACIÓN DE GENERACIÓN DE INFRAESTRUCTURAS CON IA

DOCUMENTACIÓN FRAMEWORKS PARA EL DESARROLLO DE APLICACIONES WEB Y ACCESO A DATOS

Fase 2 Del Análisis al Desarrollo de la aplicación en Laravel

Nombre del proyecto	KeyDeploy
Integrantes	Aythami Reyes Melian, Carlos Morales Perdomo, Gabriel Francisco Ruiz Bolaños, Alonso Jesús Rodríguez Betancor.
Módulo	Desarrollo Web en Entorno Servidor
Curso	2026/2027
Centro	CIFP VILLA DE AGÜIMES.
Fecha de entrega **Profesor/a**	10 de noviembre de 2026 Yeray Pérez.
Repositorio GitHub	https://github.com/Gabriel061122/KeyDeploy.git
Versión exacta del código	[Commit SHA o etiqueta ut2-entrega]
ZIP del código fuente	[UT2_Codigo_InicialesMiembrosGrupo.zip]
Presentación	[Nombre del PDF o PPTX entregado]

PLANTILLA COMÚN DE ENTREGA

No eliminar apartados. Se pueden añadir subapartados cuando estén justificados.

Cómo utilizar esta plantilla

Esta plantilla contiene instrucciones y campos de trabajo. Sustituir los textos entre corchetes por decisiones, datos, pruebas y evidencias propias. Ampliar las tablas si hacen falta filas, sin borrar apartados exigidos por el enunciado. En la portada deben indicarse el ZIP, el enlace GitHub y la versión exacta del código evaluable.

La memoria de UT2 amplía la documentación de UT1; no la sustituye. Conservar/enlazar UT1 y explicar qué cambió. Todas las afirmaciones técnicas deben corresponder con archivos, consultas, pruebas o ejecución real de la aplicación.

No copiar definiciones genéricas ni diagramas sin relación con el repositorio. No utilizar capturas como sustituto de explicaciones. No incluir contraseñas, claves o secretos. Indicar cuándo una propuesta procede de IA y cómo fue contrastada.

La entrega final será un PDF de esta memoria; la documentación técnica editable se conservará en docs/ut2/. El código fuente se entregará en un ZIP independiente y el repositorio GitHub deberá contener el mismo código, identificable mediante commit o etiqueta. La presentación se entregará por separado.

Índice del documento

1. Análisis de UT1 revisado

El registro de cambios y confirmaciones se guardará en docs/ut2/evolucion-ut1-ut2.md. Deben justificarse al menos seis cambios o confirmaciones; si algo sigue igual, también se explicará por qué.

Partir del análisis entregado en UT1 y de la corrección del profesorado. No se solicita repetir el documento original.

Documento / asunto	Antes (UT1)	Ahora (UT2)	Por qué	Efecto en código/pruebas
Datos	Producto = tabla general + una tabla de detalle por tipo (DEC-09, relación 1:1).	Tabla única products con columna config JSON (cast array) validada por tipo mediante el Enum ProductType.	Seis o más tipos con campos distintos exigirían seis tablas, migraciones, modelos y seeders más para 5 días de plazo. El JSON se valida en servidor por tipo y mantiene el formulario dinámico sin JS.	database/migrations/*_create_products_table.php; app/Enums/ProductType.php; test de alta inválida (config.vcpu). ADR-002.
Datos (región)	Región/disponibilidad : relación N:M producto × región (acordada).	Se mantiene como tabla product_availabilities con atributos propios (available, price_cents, pricing_model) y unique(product_id, region_id).	El precio depende de la región, por lo que el pivote lleva datos. Confirma la decisión de UT1.	Migración product_availabilities; CatalogTest filtro por región. ADR-001.

Documento / asunto	Antes (UT1)	Ahora (UT2)	Por qué	Efecto en código/pruebas
Módulos	Catálogo: crear, editar, retirar, consultar y filtrar (administrador).	UT2 no tiene login: el CRUD es público en desarrollo. «Retirar» = desactivar (active=false).	Sin autenticación en UT2; desactivar evita romper arquitecturas, carritos y presupuestos que referencian el producto.	ProductController@destroy; T07 comprueba active=0 con la fila conservada.
[Módulos]	[Antes]	[Ahora]	[Motivo]	[Efecto]
[Arquitectura]	[Antes]	[Ahora]	[Motivo]	[Efecto]
[Riesgos/plan]	[Antes]	[Ahora]	[Motivo]	[Efecto]

¿Qué comentario del docente cambió una decisión del grupo y cómo se ve ahora en el proyecto?

[Completar con el contenido propio del equipo y su evidencia.]

Requisito/caso UT1	Prioridad	Estado UT2 (hecho/parcial/futuro)	Evidencia o motivo
[ID y descripción]	[Alta]	[Hecho/Parcial/Futuro]	[Archivo o prueba]
[ID y descripción]	[ ]	[ ]	[ ]
[ID y descripción]	[ ]	[ ]	[ ]

2. Caso de uso propio, escenarios y decisiones de diseño

Describir un caso propio y realista de un cliente ficticio; no basta repetir el ejemplo de clase. Relacionar la necesidad con restricciones que el backend pueda verificar.

Elemento del caso	Concretar
Nombre/código y actor	[UC-xx · Actor ficticio]
Problema y objetivo	[Qué quiere lograr el cliente]
Datos de entrada	[Presupuesto, capacidad, región, necesidades…]
Precondiciones	[Qué datos/entidades deben existir]
Flujo principal	[Pasos reales de la aplicación, numerados]
Errores / casos límite	[Al menos dos, con respuesta esperada]
Resultado y evidencia	[Estado final, pantalla, test o consulta]

¿Cuáles fueron dos alternativas técnicas reales y por qué se descartó una?

Decisión	Alternativa A: ventaja / coste	Alternativa B: ventaja / coste	Elección, razón y ADR
[p. ej. disponibilidad por región]	[ ]	[ ]	[ADR-001 + prueba]

Decisión	Alternativa A: ventaja / coste	Alternativa B: ventaja / coste	Elección, razón y ADR
[otra decisión propia]	[ ]	[ ]	[ADR-002 + prueba]

Cumplimentar los seis escenarios de aceptación E1–E6. En cada caso: datos reproducibles, resultado esperado, resultado observado y prueba verificable.

ID / situación	Entrada preparada	Esperado	Obtenido	Evidencia
E1 · Flujo completo	[ ]	[ ]	[ ]	[ ]
E2 · Bloqueo	[ ]	[ ]	[ ]	[ ]
E3 · Aviso	[ ]	[ ]	[ ]	[ ]
E4 · Histórico	[ ]	[ ]	[ ]	[ ]
E5 · Formulario inválido	[ ]	[ ]	[ ]	[ ]
E6 · Transacción / fichero	[ ]	[ ]	[ ]	[ ]

3. Organización del equipo y trabajo colaborativo

Diferenciar la responsabilidad principal de cada miembro y su conocimiento del sistema completo. Justificar la rotación desde UT1.

Integrante	Responsabilidad UT1	Principal en UT2	Rotación: qué aprendió
Alonso Jesus Rodriguez Betancor	Arquitecto	Carrito, presupuesto, transacción y ficheros.	[ ]
Gabriel Ruiz Bolaños	Coordinador técnico, Analista funcional	Precios y reglas de negocio.	[ ]
Carlos Perdomo Morales	Responsable del entorno de desarrollo	Coordinador técnico, Responsable de Base de datos y catálogo.	[ ]
Aythami Reyes	Responsable de seguridad, documentación y calidad	Proyectos, requerimientos y arquitecto.	[ ]

Tarea	Autoría y apoyo	Rama / PR / review	Test/commit de evidencia
[tarea #]	[ ]	[ ]	[ ]
[tarea #]	[ ]	[ ]	[ ]

Tarea	Autoría y apoyo	Rama / PR / review	Test/commit de evidencia
[tarea #]	[ ]	[ ]	[ ]
[Añadir filas]	[ ]	[ ]	[ ]

4. Arquitectura Laravel/MVC, PHP y Blade

4.1. Visión general: responsabilidades y estructura

INSERTAR AQUÍ: Diagrama de arquitectura de la aplicación
(rutas → controladores → validación → servicios/modelos → vistas → navegador)
imagen del diagrama + enlace al archivo editable en docs/ut2/.
[Breve explicación: qué representa, por qué es así y qué prueba o archivo lo respalda.]

Componente	Responsabilidad concreta en este proyecto	Archivo/ruta que lo demuestra
Route	[ ]	[routes/web.php:…]
Controller	[ ]	[app/Http/Controllers/…]
Request / validación	[ ]	[app/Http/Requests/…]
Service / lógica	[ ]	[app/Services/…]
Model Eloquent	[ ]	[app/Models/…]
View/partial Blade	[ ]	[resources/views/…]

¿Qué lógica está expresamente fuera de Blade y por qué?

[Completar con el contenido propio del equipo y su evidencia.]

4.2. Traza real 1: petición de consulta/filtrado

Elegir una operación real de GET, filtrado del catálogo. Indicar datos reales y archivos existentes; diferenciar validación correcta y dato inválido. Adjuntar capturas legibles explicadas de ambos resultados.

Paso	Qué hace exactamente	Referencia del código / evidencia
URL + HTTP	[GET /...?... · entrada]	[ ]
Route	[Nombre/ruta y parámetros]	[ ]
Controller	[Método y argumentos]	[ ]
Validación	[Reglas y respuesta a filtro inválido]	[ ]
Model/Service/BD	[Consulta, resultado/Collection]	[ ]
View Blade	[Directiva y variable que muestra los datos]	[ ]
HTML del navegador	[Qué cambió al filtrar / sin resultados]	[Captura con explicación]

INSERTAR AQUÍ: Traza 1 dibujada con bifurcación «entrada válida / entrada inválida»

Formato recomendado: imagen del diagrama + enlace al archivo editable en docs/ut2/.

[Breve explicación: qué representa, por qué es así y qué prueba o archivo lo respalda.]

4.3. Traza real 2: operación de escritura y validación

Elegir una operación POST/PATCH/DELETE con persistencia, creación de Proyecto. Distinguir la nueva petición GET tras redirección y añadir capturas explicadas del éxito y del error de validación.

Paso	Qué sucede en caso válido	Qué ocurre ante error / referencia
Formulario/HTTP	[URL, método, campos, CSRF]	[ ]
Route/Controller	[Ruta y método llamado]	[ ]
Validación	[Reglas en Request/Form Request]	[Errores, old(), no INSERT]
Model/Service	[Cambios guardados en BD]	[No modifica BD en error]
Respuesta	[Redirect y nueva GET + vista Blade]	[Retorno al formulario + mensajes]
Comprobación	[Test / registro/consulta]	[Test y captura de error]

INSERTAR AQUÍ: Traza 2 de creación/edición con ramas éxito y validación fallida

Formato recomendado: imagen del diagrama + enlace al archivo editable en docs/ut2/.

[Breve explicación: qué representa, por qué es así y qué prueba o archivo lo respalda.]

4.4. Formulario y validación

Utilizar la información de los formularios pedidos en esta tabla, añadiendo los métodos, campos que se validan y errores:

Formulario / operación	Método y ruta	Campos/validación servidor	Evidencia éxito + error
Crear/editar Proyecto	[POST/PATCH]	[ ]	[ ]
Crear/editar/borrar requisito	[POST/PATCH/DELETE]	[ ]	[ ]
Buscar/filtrar catálogo	[GET]	[ ]	[ ]
Formulario dinámico sin JS	[ ]	[ ]	[ ]

Formulario dinámico sin JavaScript: *se permite enviar el formulario al servidor y mostrar otra página Blade con campos diferentes según el tipo de producto o requisito elegido. Debe funcionar y conservar los datos válidos al recargar.*

Referencia: *indicar ruta exacta del archivo, nombre del test o commit. Una captura sin explicación no constituye por sí sola una evidencia.*

4.5. Evidencias concretas de PHP y Blade

Usar ejemplos existentes del proyecto para demostrar el lenguaje, no solamente el framework. Para cada ejemplo indicar el archivo y explicar en pocas líneas qué hace. La comparación PHP/Blade puede ser una vista didáctica de diagnóstico.

Concepto exigido	Qué debe mostrar el equipo	Archivo o evidencia
PHP integrado frente a Blade	Un mismo resultado breve con <?php ... ?> / <?= ... ?> y con {{ ... }}; explicar el escape HTML.	[Archivo + explicación]
Variables, tipos y operadores	Variables reales; una comparación y una operación en precios o validación; explicar tipos.	[Archivo + explicación]
Condiciones y bucles	Uso justificado de @if/@else y @foreach/@forelse o equivalentes; decisiones que cambien la salida.	[Archivo + explicación]
Array o Collection	Recorrer una colección real de catálogo o requerimientos.	[Archivo + resultado]
Ámbito de variables	Dato recibido desde el Controller frente a variable local de la vista.	[Archivo + explicación]
Funciones y Request	Método propio con entradas/salida y formulario leído con Request o Form Request.	[Archivo + prueba]
Directivas de Blade	@csrf y otra directiva adicional pertinente, además de condiciones o bucles.	[Vista + explicación]
Componente o parcial reutilizable	Un fragmento Blade utilizado en dos o más pantallas.	[Dos rutas de vista]
Comentarios útiles	Comentario en el código que explique una restricción no evidente; no comentar cada línea.	[Archivo + motivo]

No situar consultas a la base de datos ni cálculos oficiales dentro de las vistas Blade.

5. Modelo de datos, integridad, migraciones y Eloquent

Entregar tres representaciones distinguibles y coherentes con la implementación: modelo E/R, modelo relacional y diagrama de tablas físicas. Sus archivos editables deben estar localizados.

INSERTAR AQUÍ: Diagrama entidad–relación revisado desde UT1

Formato recomendado: imagen exportada del diagrama + enlace al archivo editable en docs/ut2/.

[Breve explicación: qué representa, por qué es así y qué prueba o archivo lo respalda.]

INSERTAR AQUÍ: Modelo relacional (tablas lógicas, PK, FK y cardinalidades)

Formato recomendado: imagen exportada del diagrama + enlace al archivo editable en docs/ut2/.

[Breve explicación: qué representa, por qué es así y qué prueba o archivo lo respalda.]

INSERTAR AQUÍ: Diagrama de tablas implementadas en la BD

Formato recomendado: imagen exportada del diagrama + enlace al archivo editable en docs/ut2/.

[Breve explicación: qué representa, por qué es así y qué prueba o archivo lo respalda.]

5.1. Diccionario de datos y decisiones de diseño

Tabla/entidad	Atributo clave (tipo, nulabilidad, validación)	Relación PK/FK / razón
providers	id (bigint PK), name (string, único, NOT NULL)	1:N con products
regions	id (PK), code (string, único), name, country (char 2), is_eu (boolean, default false)	N:M con products vía product_availabilities
products	id (PK), provider_id (FK, NOT NULL), name (string 100), description (text, NULL), type (string, enum ProductType), billing_unit (string), config (json, NOT NULL), active (boolean, default true); índice (type, active)	FK provider_id → providers.id (restrict)
product_availabilities	id (PK), product_id (FK), region_id (FK), available (boolean), price_cents (unsigned int), pricing_model (string, enum PricingModel); unique(product_id, region_id)	Tabla pivote con datos: precio en céntimos para evitar errores de redondeo

Relación	Cardinalidad	Política al borrar/actualizar	Justificación y test
Provider ↔ Product	1:N	RESTRICT al borrar proveedor	No se puede borrar un proveedor con productos. [VERIFICAR test o intento manual]
Product ↔ ProductAvailability	1:N	CASCADE	Una disponibilidad no existe sin producto; en la práctica se desactiva el producto en lugar de borrarlo (destroy).
Region ↔ ProductAvailability	1:N	RESTRICT	No se borra una región con disponibilidades.
Product ↔ Region	N:M con atributos		Implementada con belongsToMany(...)->withPivot(available, price_cents, pricing_model). Test: filtro por región.

5.2. Eloquent, consultas y datos reproducibles

Evidencia de acceso a datos	Código/ruta/consulta	Qué devuelve y por qué es correcto
Migración PK/FK e índice	database/migrations/*_create_products_table.php (foreignId()->constrained()->restrictOnDelete(); index(type, active))	Crea la tabla con FK e índice; migrate:fresh termina en DONE
Model y relación 1:N	Provider::products(), Product::provider()	$product->provider->name devuelve el proveedor
N:M / alternativa modelada	Product::regions() belongsToMany con withPivot	Regiones de un producto con su precio
Filtro y conjunto de resultados	ProductController@index (when + whereHas)	Solo productos activos disponibles en la región elegida
Paginación o límite	paginate(9)->withQueryString()	9 por página y los filtros se conservan en los enlaces

Evidencia de acceso a datos	Código/ruta/consulta	Qué devuelve y por qué es correcto
Eager loading (evitar N+1)	with(['provider','availabilities.region'])	Evita N+1: [VERIFICAR nº de consultas con DB::listen o Debugbar, antes/después]
Seeders/factories	CatalogSeeder (determinista), ProviderFactory, RegionFactory, ProductFactory, ProductAvailabilityFactory	migrate:fresh --seed crea 2 proveedores, 5 regiones, 15 productos [VERIFICAR conteo real]

Campo / tipo especial	Enum / cast / JSON	¿Por qué no columna/relación?	Validación y prueba
config del producto	JSON / cast array	Cada tipo tiene campos distintos; seis tablas de detalle serían desproporcionadas para UT2 y el JSON admite añadir tipos sin migrar.	ProductType::configRules() exige y valida cada campo; test_alta_invalida_no_se_guarda
type del producto	Enum ProductType (string)	Lista cerrada de tipos; evita valores escritos de forma distinta.	Rule::enum en ProductRequest y CatalogFilterRequest
pricing_model	Enum PricingModel		Cast en ProductAvailability

Explicar la reconstrucción en un entorno de pruebas/desarrollo limpio y el resultado de las migraciones/seeders.

Reconstrucción en entorno limpio: php artisan migrate:fresh --seed (destruye los datos de la BD objetivo) sobre MySQL 8.4.11 y Laravel 13.35, resultado esperado: migraciones en DONE y seeder con 2 proveedores, 5 regiones, 15 productos y 6 tipos. Se comprueba con CatalogTest::test_seeder_cumple_los_minimos_del_enunciado [VERIFICAR captura de php artisan test].

6. Catálogo, proveedores, regiones, proyectos y requerimientos

6.1. Catálogo

Verificación exigida	Dato comprobado en BD	Consulta/pantalla/test que lo evidencia
2 proveedores (mínimo)	2 (NimbusCloud, Atlantic Hosting) [VERIFICAR]	Test de mínimos del seeder
5 regiones (mínimo)	5 (eu-west, eu-central, us-east, us-west, ap-south) [VERIFICAR]	Test de mínimos
15 productos/servicios (mínimo)	15 [VERIFICAR]	Test de mínimos; /products
6 tipos funcionales (mínimo)	computación, almacenamiento, bases de datos, redes, seguridad, backup	Test: distinct type >= 6

Verificación exigida	Dato comprobado en BD	Consulta/pantalla/test que lo evidencia
Producto en varias regiones	[VM Básica en eu-west, eu-central y us-east [VERIFICAR]	/products/{id}, tabla de disponibilidad
Producto no disponible en una región	VM Premium sin disponibilidad en us-east [VERIFICAR]	Filtro region_id=us-east no la muestra

Operación catálogo	Ruta/método	Qué persiste o consulta	Prueba/evidencia
Listar, detalle y filtro	GET /products, GET /products/{product}	Consulta con filtros y paginación	CatalogTest filtro [VERIFICAR]
Crear	GET /products/create, POST /products	Inserta products y product_availabilities en transacción	test_t07
Editar	GET /products/{product}/edit, PATCH /products/{product}	Actualiza producto y precios por región	test_t07
Desactivar/eliminar	DELETE /products/{product}	active=false, la fila se conserva	test_t07

Decisión obligatoria: ¿región directa en producto, tabla de disponibilidad u oferta por región? Comparar al menos dos alternativas y escoger una.

Se eligió tabla de disponibilidad por producto y región, descartando la columna region_id directa en products. Ventajas: un producto puede estar en varias regiones con precio propio y la restricción unique evita duplicados. Inconveniente: una consulta más (whereHas). Ver ADR-001.

6.2. Proyecto y requerimientos estructurados

Elemento	Modelo propio y ejemplo de BD	Validación / prueba
Proyecto	[Campos y caso ficticio propio]	[ ]
Clave de requisito	[ ]	[ ]
Operador, valor y tipo	[ ]	[Compatibilidad tipo/operador]
Obligatorio/preferencia, origen	[ ]	[ ]
CRUD de requerimientos	[Rutas y datos reales]	[ ]
Caso inválido	[Tipo/operador incompatible]	[No debe persistir]

La creación y edición de proyectos son obligatorias. El borrado se implementará cuando sea coherente con sus relaciones y se documentará el resultado; no debe dejar requerimientos o arquitecturas sin relación válida.

7. Arquitecturas, elementos, precios y reglas de negocio

7.1. Arquitectura utilizada en el proyecto

En este apartado «arquitectura» significa una combinación de servicios Cloud asociada a un proyecto, NO la arquitectura MVC interna de Laravel (ya documentada en el apartado 4). Mostrar las arquitecturas creadas por el equipo y cómo se comprueban sus configuraciones.

[Completar con el contenido propio del equipo y su evidencia.]

Documentar estos cuatro puntos con datos reales de la aplicación y evidencias que puedan repetirse:

Comprobación obligatoria	Qué debe contener la documentación	Archivo, captura o test
Arquitectura válida	Asociada a un proyecto; al menos 3 elementos de al menos 2 tipos distintos, con región, cantidad y configuración.	[Identificador + prueba]
Arquitectura rechazada	Un segundo ejemplo con incompatibilidad o requisito incumplido; mostrar el mensaje de rechazo.	[Identificador + prueba]
Varias arquitecturas por proyecto	Indicar si se admiten y cómo se guarda esa relación.	[Modelo + explicación]
Persistencia y cambios	Cómo se guardan y recuperan los elementos; qué ocurre si cambia su disponibilidad.	[Archivo + test]

7.2. Cálculo de precios: tres modelos distintos

Modalidad implementada	Tarifa/unidad/cantidad	Cálculo y redondeo	Resultado real y test
[Modelo 1]	[ ]	[ ]	[ ]
[Modelo 2]	[ ]	[ ]	[ ]
[Modelo 3]	[ ]	[ ]	[ ]

¿Qué significa el máximo mensual y cómo se separan costes únicos y recurrentes? ¿Cómo evita la aplicación confiar en importes ocultos del navegador?

[Completar con el contenido propio del equipo y su evidencia.]

Caso de precio	Datos de entrada	Resultado exacto esperado	Resultado y test
Normal	[ ]	[ ]	[ ]
Límite de cantidad/configuración	[ ]	[ ]	[ ]
Tarifa modificada	[ ]	[ ]	[ ]

7.3. Reglas de negocio: inventario obligatorio

Implementar cinco reglas como mínimo. Mas de dos deben bloquear una emisión y más de una debe advertir sin bloquear. Completar ambas tablas usando los mismos identificadores R01–R05.

Regla/propósito	Dato que lee	Condición técnica	Severidad
R01 · [ ]	[ ]	[ ]	[Error/aviso]
R02 · [ ]	[ ]	[ ]	[Error/aviso]
R03 · [ ]	[ ]	[ ]	[Error/aviso]
R04 · [ ]	[ ]	[ ]	[Error/aviso]
R05 · [ ]	[ ]	[ ]	[Error/aviso]

Regla	Mensaje y efecto en flujo	Ejemplo válido/ejemplo inválido	Test asociado
R01	[ ]	[Válido:… / inválido:…]	[ ]
R02	[ ]	[ ]	[ ]
R03	[ ]	[ ]	[ ]
R04	[ ]	[ ]	[ ]
R05	[ ]	[ ]	[ ]

¿Por qué la validación devuelve errores Y advertencias? ¿Cómo se usan para impedir o permitir un presupuesto?

[Completar con el contenido propio del equipo y su evidencia.]

8. Selección, presupuesto histórico, transacción y Storage

8.1. Selección/carrito en base de datos

Decisión	Descripción específica	Archivo/test
Persistencia sin login	[Qué representa «usuario» de desarrollo; sin prometer permisos]	[ ]
Líneas y configuraciones	[Cómo se guardan cantidad/elementos]	[ ]
Recalcular importes	[Quién calcula y cuándo]	[ ]
Carrito vacío o inválido	[Qué bloquea emisión]	[ ]
Quitar/editar líneas	[Qué cambia en BD]	[ ]

8.2. Presupuesto como snapshot histórico

Dato congelado al emitir	Cómo se conserva (líneas/JSON/mixto)	Comprobación de inmutabilidad
Nombre/tipo del elemento	[ ]	[ ]
Configuración/cantidad	[ ]	[ ]
Tarifa y periodicidad	[ ]	[ ]

Dato congelado al emitir	Cómo se conserva (líneas/JSON/mixto)	Comprobación de inmutabilidad
Totales/fecha de emisión	[ ]	[ ]

Prueba histórica: registrar tarifa inicial, emitir presupuesto, modificar catálogo y volver a consultar presupuesto. Indicar los valores antes y después.

[Completar con el contenido propio del equipo y su evidencia.]

8.3. Transacción y consistencia

Operación de emisión	Escrituras dentro de la transacción	Fallo controlado / rollback esperado
[Qué operación compuesta]	[Escritura 1, 2 y 3]	[Dónde se fuerza; qué no debe persistir]

¿Qué garantiza DB::transaction() y qué NO revierte cuando se almacena un fichero?

[Completar con el contenido propio del equipo y su evidencia.]

8.4. Subida y ciclo de vida del fichero

Aspecto	Elección del equipo	Evidencia / prueba
Entidad a la que se adjunta	[Producto/proyecto/arquitectura; razón]	[ ]
Formulario multipart, validación tipo/tamaño	[ ]	[ ]
Disco, ruta y metadatos en BD	[ ]	[ ]
Acceso/visualización	[ ]	[ ]
Sustitución y eliminación	[Cómo se evitan huérfanos]	[ ]
Fallo de BD después de escribir archivo	[Limpieza/compensación propuesta]	[ ]
Storage::fake / UploadedFile::fake	[Caso probado]	[ ]

9. Batería de tests, casos manuales, errores y seguridad

9.1. Tests automatizados

Indicar nombre REAL de cada test, nivel Unit/Feature, entrada y aserción de estado/resultado. Un «HTTP 200» aislado no basta como comprobación de integridad.

ID y prueba exigida	Unit/Feature + archivo	Entrada y aserción verificable	Resultado
T01 · Precio modalidad A	[ ]	[ ]	[Pasa/Falla + fecha]
T02 · Precio modalidad B / límite	[ ]	[ ]	[Pasa/Falla + fecha]

ID y prueba exigida	Unit/Feature + archivo	Entrada y aserción verificable	Resultado
T03 · Regla bloqueante 1	[ ]	[ ]	[Pasa/Falla + fecha]
T04 · Regla 2 / aviso	[ ]	[ ]	[Pasa/Falla + fecha]
T05 · Alta válida que persiste	[ ]	[ ]	[Pasa/Falla + fecha]
T06 · Alta inválida que no persiste	[ ]	[ ]	[Pasa/Falla + fecha]
T07 · CRUD o relación Eloquent	[ ]	[ ]	[Pasa/Falla + fecha]
T08 · Storage / archivo	[ ]	[ ]	[Pasa/Falla + fecha]
T09 · Snapshot histórico	[ ]	[ ]	[Pasa/Falla + fecha]
T10 · Transacción / fallo	[ ]	[ ]	[Pasa/Falla + fecha]

9.2. Pruebas manuales de error y aceptación

Escenario de entrada	Esperado	Obtenido y evidencia
Campo obligatorio vacío	[ ]	[ ]
Tipo/operador inválido	[ ]	[ ]
Entidad relacionada inexistente	[ ]	[ ]
Producto/región incompatible	[ ]	[ ]
Arquitectura fuera de presupuesto	[ ]	[ ]
Cantidad/configuración fuera de rango	[ ]	[ ]
Carrito vacío	[ ]	[ ]
Fichero rechazado	[ ]	[ ]
Tarifa modificada tras emitir presupuesto	[ ]	[ ]

9.3. Investigación real de un fallo

Momento	Describir con datos/archivos
Síntoma reproducible	[Entrada concreta y resultado equivocado]
Hipótesis inicial	[Explicación que pensasteis antes de revisar]
Herramienta utilizada	[Log, tinker, route:list, DevTools, test...]
Causa confirmada	[Por qué sucedía]

Momento	Describir con datos/archivos
Corrección aplicada	[Archivo y commit/PR]
Test que inicialmente fallaba	[Nombre y salida relevante, después verde]

9.4. Seguridad básica e integridad

Control que se ha comprobado	Cómo se verifica (ruta, test, revisión)
CSRF en formularios	[ ]
Validación servidor	[ ]
Escape de salida HTML	[ ]
Protección de asignación masiva	[ ]
Integridad de FK y existencia	[ ]
Ausencia de .env y secretos en Git/ZIP	[ ]

10. Uso crítico de inteligencia artificial

Si se utilizó IA en el desarrollo, registrar al menos tres decisiones relevantes y cómo se verificaron. Si no se utilizó, documentar tres decisiones técnicas propias con alternativas y pruebas; no se penaliza decidir no utilizar IA. Antes de consultar una herramienta: necesidad, restricciones, propuesta propia y alternativa. No adjuntar conversaciones privadas completas ni datos sensibles.

ID / problema y propuesta propia	IA consultada (si procede) / alternativas	Aceptado, modificado o descartado, y por qué	Prueba, autor y PR
DEC-01 · [ ]	[ ]	[ ]	[ ]
DEC-02 · [ ]	[ ]	[ ]	[ ]
DEC-03 · [ ]	[ ]	[ ]	[ ]

Explicar una recomendación de IA que se rechazó por errónea, compleja o fuera de UT2; o una decisión propia sin IA que se contrastó.

[Completar con el contenido propio del equipo y su evidencia.]

Registrar un caso límite que la primera solución no resolvía y qué test o modificación lo solucionó.

[Completar con el contenido propio del equipo y su evidencia.]

¿Qué parte del resultado fue verificada en documentación, migraciones, ejecución o test?

[Completar con el contenido propio del equipo y su evidencia.]

11. Riesgos, limitaciones, módulos futuros y conclusiones

Módulo / asunto	Estado al cierre UT2	Riesgo o limitación verificable	Continuidad UT3/UT4
Autenticación/usuarios	[No exigido en UT2]	[ ]	[ ]
API REST/servicios	[No exigido en UT2]	[ ]	[ ]
IA integrada	[No exigido en UT2]	[ ]	[ ]
Núcleo catálogo/proyectos	[Hecho/parcial]	[ ]	[ ]
Presupuesto/Storage/tests	[Hecho/parcial]	[ ]	[ ]

El README del repositorio debe permitir instalar el ZIP, crear .env desde .env.example, preparar la base de datos con migraciones y seeders, arrancar la web y ejecutar las pruebas. Indicar qué módulos de UT2 funcionan y cuáles quedan para UT3/UT4.

Conclusiones del equipo: ¿qué decisión cambiaríais hoy y qué evidencia respalda esa reflexión?

[Completar con el contenido propio del equipo y su evidencia.]

Integrante	Aportación verificable	Qué aprendió y qué parte puede defender
[Nombre]	[Issue/PR/test/documento]	[ ]
[Nombre]	[Issue/PR/test/documento]	[ ]
[Nombre]	[Issue/PR/test/documento]	[ ]
[Añadir]	[ ]	[ ]

Anexo A. Ficha de decisión de arquitectura (ADR)

Duplicar esta ficha por cada decisión que quiera documentarse; al menos las que justifican el diseño propio, la región y los precios.

Campo	Contenido a completar
ID / título / fecha	ADR-001 · Disponibilidad y precio por región · 09/11/2026
Contexto y necesidad	El enunciado exige productos en varias regiones y otros no disponibles en alguna, con precio relacionado con la región.
Alternativa A	Columna region_id en products. Ventaja: sencilla. Coste: un producto por región (duplicados) y no admite precio por región sin duplicar.
Alternativa B	Tabla product_availabilities (producto, región, available, price_cents, pricing_model). Ventaja: N:M con datos y una fila por región. Coste: join/whereHas.
Decisión tomada	B, por integridad (unique) y porque encaja con la decisión N:M de UT1.
Consecuencia/limitación	[Riesgo asumido o coste futuro]
Evidencia técnica	Migración product_availabilities; filtro por región en CatalogTest. [Migración, prueba, captura, PR]

Campo	Contenido a completar
Participantes y revisión	Propone Carlos, revisa Gabriel (precios) y Alonso (PR) **[VERIFICAR]**

Campo	Contenido a completar
ID / título / fecha	ADR-002 · Detalle de producto como JSON (config) · 09/11/2026
Contexto y necesidad	UT1 (DEC-09) planteó una tabla de detalle por tipo; en 16.3 quedó pendiente decidir la implementación.
Alternativa A	Una tabla de detalle por tipo (1:1). Ventaja: columnas tipadas. Coste: 6 o más tablas, migraciones, modelos y seeders más.
Alternativa B	Columna config JSON con cast array y validación por tipo en el Enum. Ventaja: rápida y extensible. Coste: sin tipado de BD ni FK en los campos, consultas por campo más limitadas.
Decisión tomada	B: validación en ProductRequest con ProductType::configRules().
Consecuencia/limitación	Filtrar por un campo de config (p. ej. RAM) requeriría consultas JSON; se deja para UT3/UT4.
Evidencia técnica	products.config (json); test_alta_invalida_no_se_guarda. [Migración, prueba, captura, PR]
Participantes y revisión	Propone Carlos, revisa el equipo **[VERIFICAR]**.

Anexo B. Inventario de rutas, diagramas y evidencias

ID	Ruta/artefacto exacto	Para qué sirve	Dónde se explica en memoria
EV-01	[routes/web.php:línea, URL + método]	Traza HTTP	4.2
EV-02	app/Http/Controllers/ProductController.php, app/Http/Requests/CatalogFilterRequest.php	Traza HTTP	4.3
EV-03	database/migrations/*_create_{providers,regions,products,product_availabilities}_table.php	ER y FK	5
EV-04	tests/Feature/CatalogTest.php (T07, filtro, mínimos)	Test relevante	9
EV-05	database/seeders/CatalogSeeder.php	Datos reproducibles	5.2 y 6.1

Criterios de corrección de UT2

Las tres partes del proyecto se califican por separado de 0 a 10. Parte A (código): 40 % del módulo; Parte B (documentación): 20 %; presentación y defensa: 10 %. En conjunto representan el 70 % de la nota final de la UT.

La rúbrica detallada del código está en el documento «UT2 · Parte A. Código fuente, requisitos y rúbrica». Esta plantilla contiene las rúbricas de la memoria y de la presentación para evitar confundir qué se programa con qué se explica.

Criterios de corrección: Documentación (10 puntos)

La memoria debe demostrar lo que realmente existe en el proyecto, con referencias a archivos, diagramas, consultas, pruebas o capturas explicadas. Las tablas deben ampliarse cuando sea necesario.

Criterio que se califica	Máximo	Qué debe demostrarse
Revisión de UT1	1,00 puntos	Seis cambios o confirmaciones justificados, relación con la corrección docente y efectos en código/pruebas.
Caso propio y escenarios	0,90 puntos	Caso propio, alternativas y seis escenarios E1–E6 con entrada, resultado y evidencia.
Organización del equipo	0,60 puntos	Rotación de funciones, tareas, revisiones y aportaciones verificables.
MVC y dos trazas HTTP	1,10 puntos	Diagramas correspondientes al código real, archivos concretos, recorrido correcto y validación fallida.
PHP y Blade	1,00 puntos	Evidencias reales de variables, tipos, funciones, bucles, condicionales, directivas y componentes reutilizables.
Modelos y Eloquent	1,20 puntos	Tres diagramas coherentes, diccionario, relaciones y justificación de consultas/migraciones.
Catálogo, proyectos y arquitecturas	1,00 puntos	Decisiones y pruebas de datos, requerimientos y arquitecturas Cloud válidas e inválidas.
Precios y reglas	1,00 puntos	Tres modalidades, resultados comprobables y cinco reglas con ejemplos, mensajes y tests.
Carrito, presupuestos, transacción y Storage	0,90 puntos	Cómo se guardan los datos, cómo se conserva el presupuesto y cómo se gestionan los ficheros.
Pruebas, fallo y seguridad	0,90 puntos	Diez tests identificados, pruebas manuales, investigación de un error y controles de seguridad.
Decisiones, IA y cierre	0,40 puntos	Decisiones propias o IA contrastada, ADR, limitaciones, continuidad, referencias y aportación individual.
TOTAL	10,00 puntos	Calificación de esta parte sobre 10.

2. Criterios de corrección de presentación y defensa (10 puntos)

Criterio que se califica	Máximo	Qué debe demostrarse
Material y organización de la presentación	1,50 puntos	Archivo entregado y explicación clara, breve y bien estructurada.
Demostración funcional de la aplicación	3,00 puntos	E1 completo con datos guardados: catálogo, proyecto, requisitos, arquitectura, precios, reglas, carrito y presupuesto.

Criterio que se califica	Máximo	Qué debe demostrarse
Demostración de errores y pruebas	1,50 puntos	Se muestra al menos un bloqueo, una advertencia, un error de formulario y la conservación del presupuesto.
Modelo de datos y decisiones técnicas	1,00 puntos	Se explica una decisión propia y su alternativa, con evidencias del código.
Participación del grupo	1,00 puntos	Todos los integrantes intervienen y explican alguna parte del trabajo.
Respuestas individuales en la defensa	2,00 puntos	Cada integrante localiza, explica o modifica código sencillo y responde a preguntas acordes a UT2.
TOTAL	10,00 puntos	Calificación de esta parte sobre 10.

3. Cómo se calcula la nota del proyecto

Nota independiente	Peso en el módulo	Qué se evalúa
A · Código fuente (0–10)	40 %	Funcionalidad, datos, pruebas, ZIP ejecutable y GitHub.
B · Documentación (0–10)	20 %	Memoria UT2, decisiones, diagramas y evidencias reales.
P · Presentación (0–10)	10 %	Material, demostración y defensa.
TOTAL	70 %	El otro 30 % corresponde a las demás actividades del módulo.

Aportación de UT2 a la nota final del módulo = (A × 0,40) + (B × 0,20) + (P × 0,10). Esta aportación puede sumar hasta 7 puntos sobre 10.

Si necesitas la calificación interna del proyecto sobre 10 antes de aplicar el 70 %, utiliza: Nota del proyecto = (4 × A + 2 × B + P) ÷ 7.